

AI Agent Workflow Builder — Architectural Overview

System Overview

The AI Agent Workflow Builder is a low-code platform that lets users compose agentic workflows from typed steps (LLM calls, HTTP requests, DB writes, notifications, conditional branches, and approval gates). Workflows belong to an organization; each user's rights are scoped per-organization via the `org_members` junction table (roles: owner, editor, viewer). Execution is managed by two privileged GraphQL Actions (`triggerWorkflowRun` and `approveStep`) that enforce business rules unavailable to declarative Hasura permissions.

Core Data Models

The system revolves around six primary entities (simplified):

```
erDiagram
    ORGANIZATION |||o{ ORG_MEMBERS : "has"
    ORGANIZATION |||o{ WORKFLOWS : "owns"
    ORG_MEMBERS ||..|| USERS : "member"
    WORKFLOWS |||o{ WORKFLOW_STEPS : "contains"
    WORKFLOWS |||o{ WORKFLOW_TRIGGERS : "has"
    WORKFLOWS |||o{ WORKFLOW_RUNS : "initiates"
    WORKFLOW_RUNS |||o{ STEP_RUNS : "executes"
    WORKFLOW_STEPS ||..|| STEP_RUNS : "template for"

    ORGANIZATION {
        string id PK
        string name
        string slug
        int quota_limit
        int quota_used
        string quota_period_start
    }
    ORG_MEMBERS {
        string org_id PK,FK
        string user_id PK,FK
        string role "owner|editor|viewer"
    }
    WORKFLOW {
        string id PK
        string org_id FK
        string name
        string description
        boolean is_active
        string created_by
        string created_at
        string updated_at
    }
    WORKFLOW_STEP {
        string id PK
        string workflow_id FK
        string org_id FK
        int position
        string name
        string type
        "llm_call|http_request|db_write|notify|conditional_branch|approval_gate"
        jsonb config
        int max_attempts
        string on_true_step_id FK
        string on_false_step_id FK
    }
    WORKFLOW_TRIGGER {
```

```

        string id PK
        string workflow_id FK
        string type "manual|webhook|scheduled|database_event"
        boolean is_enabled
        jsonb config
        string webhook_token
        string cron_expr
        string next_run_at
    }
    WORKFLOW_RUN {
        string id PK
        string workflow_id FK
        string org_id FK
        string trigger_id FK
        string trigger_type "manual|webhook|scheduled|database_event"
        string triggered_by
        string status "pending|running|paused|succeeded|failed"
        jsonb input
        string error
        string started_at
        string finished_at
        string created_at
    }
    STEP_RUN {
        string id PK
        string workflow_run_id FK
        string step_id FK
        string org_id FK
        int position
        string name
        string type
        "llm_call|http_request|db_write|notify|conditional_branch|approval_gate"
        jsonb config
        string status
        "pending|running|awaiting_approval|succeeded|failed|skipped"
        jsonb input
        jsonb output
        string error
        int attempt
        int max_attempts
        string approved_by FK
        string approved_at
        string approval_note
        string started_at
        string finished_at
    }
}

```

Note: The `org_id` is denormalized on steps, runs, and triggers to simplify row-level security joins.

Schema Reasoning

1. **Per-organization scoping** – Every table that is user-visible (workflows, workflow_steps, workflow_triggers, workflow_runs, step_runs) either stores an explicit `org_id` or can reach it via a foreign-key chain (`workflow.org_id`). This enables a single row filter pattern: `org.members.user_id = X-Hasura-User-Id`.
2. **Denormalization for performance** – `org_id` on workflow_steps, workflow_triggers, workflow_runs, and step_runs avoids extra joins in the permission filter while keeping data consistent via triggers/application logic.
3. **Immutable run state** – workflow_runs and step_runs are never mutated by the user role; only the privileged Actions may insert/update them. This eliminates a class of privilege-escalation where a viewer could directly set `status = 'approved'`.
4. **Step-type extensibility** – The type column uses an open enum; new step types can be added without schema changes, only requiring updates to the Action executor and UI.
5. **Approval gate modeling** – An approval_gate step is a regular step; its

`step_run.status` transitions to `awaiting_approval` when the runner reaches it. The `approveStep` action flips it to `approved` and resumes the run.

Permission Layers

Layer1 – Declarative Row-Level Security (Hasura)

All tables expose only the `user` role (logged-in) and `public`. Row filters are expressed as joins to `org_members` keyed by the session variable `X-Hasura-User-Id`. Examples:

- `workflows.select`: { `org`: { `members`: { `user_id`: {`_eq`: `X-Hasura-User-Id`} } } }
- `workflow_steps.insert`: combines `org/role` check with a declarative step-type gate (e.g., forbid `db_write` unless the caller is an *owner* of the workflow's `org`).
- `workflow_runs / step_runs`: **no** `select/insert/update` permission for the `user` role – they are readable only via subscriptions (which inherit the same `org` filter) and mutable exclusively by the `admin-secret` Actions.

Because the filter re-evaluates the join on every request, a user cannot “guess” an ID from another organization; the filter returns an empty set, not an error.

Layer2 – Procedural Guard in Action Handlers

Logic that cannot be expressed as a static row filter lives in the two Nhost Actions:

- **triggerWorkflowRun**
 1. Resolves `x-hasura-user-id` from the verified JWT.
 2. Queries `org_members`  `workflows` to confirm membership and fetch the caller's role.
 3. Rejects viewers (403) and enforces quota by checking organizations.
 4. Inserts a `workflow_run` (`status=running`).
 5. Executes steps in order, inserting `step_run` rows, updating them with outputs/errors.
 6. When an `approval_gate` step is encountered, the run is paused (`workflow_run.status = 'paused'`) and the handler returns; the `step_run` remains `awaiting_approval`.
 7. If the run finishes without pausing, quota is incremented and the run marked completed.
- **approveStep**
 1. Resolves caller's `x-hasura-user-id`.
 2. Loads the target `step_run` and its parent `workflow_run` via an admin query.
 3. Re-queries `org_members` to verify the caller is an owner/editor of the workflow's organization.
 4. Validates that the step is indeed an `approval_gate` and its status is `awaiting_approval`.
 5. Updates the `step_run` (`status='approved'`, `approved_by`, `approved_at`).
 6. Sets the parent `workflow_run` back to `running`.
 7. Resumes execution (either inline or via a queued job) – the same loop that `triggerWorkflowRun` uses continues from the next step.

Both actions **never trust role information supplied by the client**; they always derive it from a fresh database lookup, guaranteeing that a compromised token cannot be used to impersonate a higher role in another organization.

Approval-Gate Pause/Resume Mechanics

1. **Pause** – During `triggerWorkflowRun`, after a step of type `approval_gate` is executed (its `step_run` is inserted with status `running` then immediately updated to `awaiting_approval`), the handler updates the parent `workflow_run.status` to `'paused'` and records `paused_at_step_run_id` (exposed via the GraphQL result). The handler then exits, leaving the run suspended.

2. **Resume** – When a user calls `approveStep` with the `step_run_id` of the paused gate:

- The action validates ownership and that the step is an approval gate awaiting approval.
- It updates the `step_run` to `approved` (with timestamps and approver).
- It sets the parent `workflow_run` back to `'running'`.
- The execution loop is re-entered (either by re-invoking the same Action with a continuation token or via an internal background worker) which reads the next pending step and proceeds until completion or another pause.